MoneyMind

Product design case study report

Portfolio prototype · 22 October 2026

Design, writing, and a working browser prototype are complete for review. No customer interviews or usability sessions have been run. Problems, personas, and success targets in the project remain hypotheses until that research happens.

This report summarizes the files in 01-MoneyMind/. It does not replace them. Claims below are labeled as verified in the prototype, documented from a cited source, assumed, or still requiring validation.

1. Executive summary

Product. MoneyMind is a mobile-first web prototype. A person enters a current balance, expected income, and upcoming bills. The app builds a 30-day cash estimate and a safe-to-spend figure, then shows the math.

Unique value proposition. Know when you may run short of money before it happens. The interface keeps that promise only if the low point is visible, the items that created it can be inspected, and the number is named as an estimate whenever it is used to decide.

Problem and solution, as a hypothesis. A checking balance says what is available now. It does not say which bills are already promised before the next deposit. MoneyMind answers a different question: if this amount is spent today, what is the lowest estimated point in the next 30 days, after a set-aside the person chooses?

Implementation status. The information architecture, annotated wireframes, design system, high-fidelity specifications, and browser prototype are built. Research instruments and a usability plan are written. User validation has not started. Forecast arithmetic is unit tested. The primary browser path was exercised with headless Chrome on 22 October 2026.

Differentiator, documented and not yet user-proven. The product shows a dated trough built from typed items, with the formula on screen, and it does not require a bank connection. Competitor pages already offer leftover figures, projected cash flow, or zero-based budgeting. MoneyMind does not claim those ideas are new. Sources were checked on 22 October 2026 and are listed in 01-product-analysis/competitive-analysis.md.

2. Product brief and objectives

Target users

No segment has been confirmed with interviews. The brief states two hypotheses. The primary sketch is an adult who manages month-to-month cash and wants to avoid a surprise before a bill or between paydays. The secondary sketch is a person with irregular income who can still enter the dates and amounts they expect.

Out of scope: professional traders, financial advisers, business accounting teams, shared household ledgers, and anyone whose main job is investing, lending, or credit scoring. See 01-product-analysis/product-brief.md and 02-ux-research/personas.md.

Job to be done

The only job treated as the design brief comes from the product requirements: when deciding whether they can afford a purchase, the person wants to understand upcoming commitments and the likely balance. Related jobs that the prototype does support are updating a moved bill, leaving an in-app alert, and exporting or deleting the local record. Category history, shared spending, and advice are refused. See 02-ux-research/jobs-to-be-done.md.

Goals

- Make the low point and the safe-to-spend math understandable.
- Let a first useful estimate be created without a bank connection.
- Keep every decision figure labeled as an estimate.
- Make manual entry short enough that a reviewer can reach a plan, with labeled sample data available.

Scope and constraints

Must-have behavior from the requirements is in the prototype: onboarding that states limits, a manual balance and items, a labeled 30-day estimate, the safe-to-spend explanation, add/edit/delete, an in-app low-balance

alert, and empty, error, and stale states. Should-have behavior is also built: an unsaved what-if, categories that do not change the estimate, and a local JSON export.

Not built, on purpose: read-only bank linking, household budgets, savings goals, and an income range. Payments, transfers, lending, credit scores, investing, and product recommendations are non-goals.

- Calculations run in the browser. The prototype is a static site.
- Storage is localStorage under moneymind.v1. It is disclosed as unencrypted and is not acceptable for production financial data.
- One currency symbol per plan. The display list is USD, CAD, GBP, EUR, and AUD. There is no foreign exchange.
- A device notification can be sent only while the page is open and only after permission. There is no background schedule.
- The accessibility target is WCAG 2.2 AA for the screens that exist. That is a design target, not a certificate.

3. Product analysis and rationale

Position

For someone deciding whether to spend today, MoneyMind is the 30-day cash estimate that shows the lowest point and the math behind safe to spend. It is for people willing to enter their own numbers. The category is a personal cash-timing tool. The brief keeps it apart from account aggregation, envelope budgeting, calendar-month leftovers, long-range net-worth forecasts, and coaching or product offers. See 01-product-analysis/uvp-and-positioning.md.

Alternatives reviewed

The competitive set was limited to pages and help articles opened for this project. Access date: 22 October 2026. A feature is not called missing unless a cited source says so. Prices that were not opened on the vendor’s pricing page are marked unverified in the source file. Accessibility of competitor products was not audited.

Alternative	What the cited sources support	Documented difference
PocketGuard	Homepage: Leftover / what is safe to spend after necessities; Pace forecasts from history; read-only bank linking via Plaid, Finicity, and Salt Edge. Public price was not on the fetched homepage and was not re-verified.	A dated 30-day trough from typed items, without a bank link. PocketGuard already promises a safe-to-spend idea.
Quicken Simplifi	Product page: projected balances and what-if language; promo $3.99 versus $6.99 per month, billed annually. Spending Plan help (updated 11 June 2026) excludes existing balances. Projected Cash Flow help (updated 16 October 2026) excludes Planned Spend.	Balance, bills, and planned expenses sit on one 30-day estimate.
Copilot Money	Cash flow looks backward and excludes unpaid recurrings. Free to Spend is budget remaining. Recurring income is not supported. The low-balance alert described is a fixed $100 on the account. Homepage content: $95 a year or $13 a month. Android was neither confirmed nor denied.	Future-dated income and expenses, including a projected trough.
Monarch Money	Forecasting help (updated 20 October 2026): long-horizon net worth and cash flow, marked as a Plus feature. The Plus price was not opened.	The next 30 days of entered cash items, for a purchase decision.
YNAB	Pricing page: $109 a year or $14.99 a month. Zero-based method: assign cash already on hand. Expected income is not treated as spendable.	Future income stays visible and is excluded from safe to spend when it arrives after the low point.
Substitutes	A spreadsheet, a calendar, and a bank balance are real workflows. No specific bank app’s bill-pay forecast was audited.	The explanation has to be clearer than a spreadsheet the person already trusts.

Decisions and why they were made

- Safe to spend is the lowest estimated balance in the next 30 days, minus a set-aside the person chooses. It is defined as a purchase decision, rather than “what is left this calendar month.”
- On any day, expenses are counted before income, so a paycheck on the same date does not hide the dip.
- Income after the low point stays on the timeline and is excluded from safe to spend.
- The result is one point estimate. A probability band was refused because no variance model exists.
- The set-aside may be zero. The form does not recommend a cushion, because a recommended percent would read as advice.

- A what-if is visually separate and is written into the plan only after Add to plan.
- The low-balance alert always has an in-app state. The prototype does not claim a notification after the page closes.
- Sample data is labeled and stays labeled until the person removes it.

Prioritization

Priority follows the requirements, then a second cut for what had to be on screen to make the value proposition inspectable. Effort notes in 03-information-architecture/feature-prioritization.md are judgments about this prototype, not a team estimate. The written cut line is: keep Home, the explanation, add/edit/delete, and the timeline table. Cut the chart before the table. Cut categories before what-if. Do not cut the disclaimer to save space.

Assumptions and risks

Confidence ratings in 01-product-analysis/assumptions-and-risks.md are design judgments. None have been confirmed with participants. The highest-risk assumptions are that the job is a near-term shortfall rather than last month’s spending, and that people will type a balance and a few dated items. If either is wrong, the information architecture is aimed at the wrong task.

Product risks called out in the requirements, and the responses already in the prototype: false confidence is met with the word Estimate, a standing disclaimer, and the full formula; manual entry is met with two-step onboarding, a preview before save, and labeled sample data; sensitive data is met by refusing bank credentials and saying that storage is unencrypted; recurrence mistakes are met by an anchor-day monthly rule that is unit tested. Those responses are untested with users. The guardrail “fewer than 10 percent believe the forecast is guaranteed” is a target, not a result.

4. UX research and evidence

Methods actually used

Completed work is a desk review of the competitor sources above, plus the product requirements. The research plan, screener, and interview guide are written and unused. The plan calls for semi-structured interviews of about 45 minutes, focused on past behavior, and it says not to pitch MoneyMind in the interview. No participants have been recruited. See 02-ux-research/research-plan.md.

Verified findings

There are no interview findings. 02-ux-research/research-findings.md is an empty template: no observations, no interpretations, and no quotes. Competitive observations in section 3 are the only external evidence, and they are limited to the pages that were opened.

Implications that are already designed, and still assumed: if people solve timing with a calendar, the timeline has to be faster than that calendar; if people refuse to type bills, sample data is only a demo and the “useful without a bank” claim fails for them; if the pain is month-end category guilt, the safe-to-spend formula is the wrong headline; if any future number is heard as a promise, labels alone are an insufficient guardrail. Those implications are written as things a later study would have to check.

Proto-personas

Hypothesis A, payday-gap planner: paid every two weeks, with at least one large monthly bill, checking a phone balance before an ordinary purchase. The sample plan (rent before the next payday, a small set-aside, transit, and groceries) is an illustrative story for this sketch. Hypothesis B, irregular-income planner: deposits arrive on uneven dates, and the person can name the next expected deposit. The prototype accepts that deposit as dated income and still shows a point estimate. It does not invent a low and a high from history. Ages, cities, income amounts, and invented names were omitted on purpose. See 02-ux-research/personas.md.

Journey

02-ux-research/journey-map.md is marked Hypothesis. It sketches five stages for Hypothesis A: payday lands, an ordinary expense appears, the bill posts, the gap before the next pay, and the next payday. The designed opportunity is to ask for the balance with its date, put small items on the same timeline as rent, and compute safe to spend from the future low point so the later paycheck does not inflate it. There are no quotes on the map.

Outstanding research questions

- When people last decided whether they could spend, what did they look at, and what happened afterward?
- How do they track bills that fall between paydays today?
- Where have they been wrong about money that looked available?
- What would they need to see before they trusted a forward-looking number?
- How many items are they willing to enter before a tool feels useful?
- Do irregular-income and regular-payday situations fail in the same way?

5. Information architecture and user flows

MoneyMind is one plan in one browser. There is no account switcher and no marketing site inside the prototype. The source map is 03-information-architecture/sitemap.md.

Navigation

Below 880px, Home, Timeline, What-if, and Plan sit in a bottom bar. Settings is in the header. From 880px, the same destinations, including Settings, sit in a left column, and the header Settings link hides so it is not duplicated. Onboarding, balance editing, the explainer replay, privacy, and the unreadable-data screen hide the app bar and each offer a text path back.

Labels were chosen to avoid promise language. Home is the decision screen. Timeline is the 30-day ledger. Plan is the list of entered items, because nothing has posted. Global chrome carries an Estimate pill, a Sample pill when the plan is sample data, and either “On this device” or “Offline.”

Primary journey

Can I afford this? Open Home, read safe to spend and the low-point date, open the calculation, see which items were counted and which income arrives later, try a purchase, then discard it or add it to the plan. The end state is that the person can say whether the purchase is still a scenario, and the home number matches that choice. Sample data is an allowed start for a demo when the banner is visible. See 03-information-architecture/user-flows.md.

Alternate paths

- First estimate: the explainer, then either labeled sample data or a balance, date, currency, and set-aside, then items. Invalid fields stay on the form with a message.
- Edit or delete: detail, edit with a preview of the new low point, or delete with a confirm dialog. Cancel is focused first. One undo restores the item.
- Alert: set an amount. If any estimated day is strictly below it, the in-app status names the first date. Permission denied or unavailable is explained, and the in-app alert remains.
- Recovery: a balance older than seven days raises a stale banner and is not rolled forward. Unreadable storage is kept until the person downloads the raw text and confirms erasure. A future balance date is rejected. An item with a bad amount is rejected with a message that says how to fix it.

6. Wireframes and UI design

Screens

Twelve annotated grayscale wireframes use a 390 × 844 frame on a larger annotation sheet. They live in assets/wireframes/wf-01 through wf-12. Further screens are specified and built without a separate sheet. High-fidelity captures are viewport screenshots in assets/screenshots/, taken on 22 October 2026 after a narrow-width overflow fix. They are not full-page scrolls. There is no Figma file.

ID	Screen	Role
WF-01	Onboarding	States limits before any number is asked for.
WF-02	Starting balance	Balance, date, currency symbol, and set-aside.
WF-03	Home	Safe to spend, low point, and the next items.
WF-04	Timeline	Ledger, sparkline, and a day table.
WF-05	Explanation	Formula, counted items, and later income.
WF-06 to WF-07	Add, edit, detail	Entry, preview, and one item.
WF-08	What-if	Scenario beside the saved estimate, marked not saved.
WF-09 to WF-10	Alert and settings	Threshold, export, delete, and privacy.

ID	Screen	Role
WF-11 to WF-12	Empty and unreadable data	No items yet; corrupt storage is not silently wiped.
WF-13 to WF-19	Plan, summary, how, privacy, loading, offline, stale	Built in the prototype. Loading is a preview because storage reads are synchronous.

The two captures below are the decision screen and the formula. Sample figures are illustrative. Full files: assets/screenshots/02-home-mobile.png and 05-explain-mobile.png.

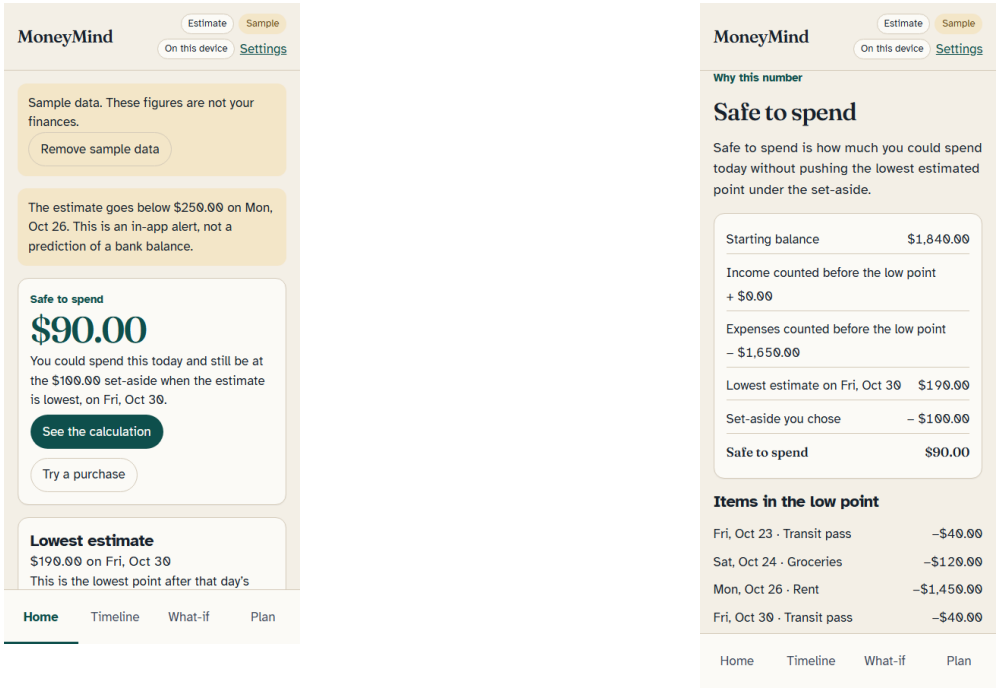


Figure. Left: Home, with the sample banner kept in frame. Right: the explanation, where each row ties to a starting balance, an entered item, or the chosen set-aside.

Layout and interaction decisions

- Show the dip beside the headline number. A calm total that hides the shortfall date fails the job.
- Every explanation row maps to something the person typed. A row that cannot be tied to an input does not belong.
- Uncertainty is a label. The interface does not add a confidence percentage or a shaded range.
- Shortfalls are dates and amounts. The copy does not scold, and a positive number is not treated as a prize.
- Scenarios, previews, and state previews say they are not part of the plan. Delete asks. The destructive button is not the one focused first.
- The chart is a glance. The list and the table are the record. The lowest day is also named in text.

Visual direction and components

The direction is named Clear ledger: a paper ledger in readable type, for someone who may be anxious. The palette is warm paper (#F3EFE6), ink, teal (#0E4F4C) for actions, and rust (#7A2E2A) only when the estimate is short of the set-aside, always with words. Sample uses an amber band. Type is Atkinson Hyperlegible for UI text and Fraunces for the decision figure, with Segoe UI and Georgia as fallbacks. Components are semantic elements in app/src/main.ts and app/styles.css, not a separate library. Tokens, contrast pairs, and component notes are in 05-ui-design-system/.

Responsive behavior and accessibility

The UI is one column below 880px and gains side navigation from 880px. Touch targets for buttons, navigation, and fields are at least 44px. Documented token pairs were calculated above 4.5:1 for text. The chart has an accessible name, and a table repeats every day. Dialogs are labelled, trap Tab, and close on Escape. prefers-reduced-motion is honored. The theme is light only; a dark theme was not shipped unchecked.

Known limits, from 05-ui-design-system/accessibility.md: no VoiceOver, NVDA, or keyboard-only participant session; native date inputs vary by browser; 400% zoom was not manually verified; this is not a conformance claim.

7. Implementation and prototype

Features in the running prototype

The app is served from app/ on 127.0.0.1:4173. Source is TypeScript in app/src/, compiled to app/dist/. There is no UI framework. Routes are hash URLs. The interface is built with DOM APIs. npm test typechecks and runs the forecast tests. npm start compiles, then serves.

Behavior	Where	What the prototype actually does
Onboarding and balance	#/welcome, #/start, #/balance	States limits, then saves a balance, date, currency symbol, and set-aside. Future dates are rejected.
Items	#/plan, #/items/…	Add, edit, and delete income and expenses. Repeats: once, weekly, biweekly, monthly. Monthly uses the anchor day and clamps only in shorter months.
Estimate	#/home, #/timeline, #/explain	Thirty calendar days from the balance date. Expenses before income on the same day. Safe to spend can show $0 with a separate shortfall.
What-if	#/what-if	An unsaved scenario in its own column. Discard clears it. Add to plan writes a real item.
Alert	#/reminders	In-app status when any day’s low is strictly below the chosen amount. Equal is not below.
Summary and export	#/summary, Settings	Category totals for the window. They do not change the estimate. Export downloads a local JSON file.
Trust states	Banners, #/error, #/privacy	Sample, stale, empty, offline, and unreadable storage. Corrupt data is not erased until confirmed.

Illustrative calculation

The worked example in the tests and in the sample plan uses a balance of $1,840 and a set-aside of $100, with rent before payday. When the dates line up with that fixture, the lowest estimate is $190 and safe to spend is $90. A $150 expense on the date used in the what-if test drops the low point and produces a $60 shortfall, shown as $0 spendable and $60 short. This is a hand-checked fixture, covered by app/test/forecast.test.ts. It is not a user result and not a guarantee.

Architecture

Money is integer cents. Dates are calendar dates, so a daylight-saving change cannot shift a day. Occurrences before the balance date are not applied again. The forecast starts on the saved date and does not roll an old balance forward. A balance older than seven days is marked stale. The saved record includes a derived forecast snapshot labeled “Derived estimate. Not a guarantee.”

Real, local, and simulated

- Real in the browser: forecast math, form validation, local save, export file creation, and the Notification API call when permission is already granted and the page is open.
- Local and disclosed: persistence in this browser only. There is no account and no upload.
- Labeled fiction: sample data, and the preview routes for loading, error, empty, and offline, which do not change the saved plan.
- Intentionally absent, and not faked: bank linking, payments, background notifications, a service worker, analytics, and encryption.

Known limits

From 07-prototype/prototype-review.md and VALIDATION.md: one currency and no conversion; one cash pool; no income range; sample data stays labeled even if edited; focus management was checked in code and in the smoke script, not with a screen reader; screenshots are viewport captures. FR-06 is partial because the alert cannot fire after the page closes. A legal retention policy is not written. Countries beyond the display-symbol list are undecided.

8. Testing and validation

Tests performed on 22 October 2026

npm test in app/ passed 18 deterministic forecast tests after the TypeScript conversion. The suite covers the rent-before-payday fixture, same-day ordering, a visible shortfall, a negative starting balance, the earliest low point when values tie, monthly clamping, occurrences before the window, an item outside the window, an unsaved what-if that does not mutate the plan, stable output for the same inputs, an edit that changes the low point, a strict below-threshold check, stale and coverage helpers, category totals, amount parsing, labeled sample data, storage round-trip and corrupt data, and rejected calendar dates.

Headless Chrome ran app/test/smoke.html against the compiled bundle. Logged checks passed for onboarding, the sample home, the explanation, the timeline table and chart, what-if columns, item validation, save, reminder copy, the export control, a stale balance, and a sample link that stays on the requested screen. The same run checked that onboarding, home, the explanation, the timeline, and the sample explanation fit a 390px frame without horizontal scrolling. The smoke script does not cover undo, the contents of an export file, or a screen reader. Keyboard order was not walked by a person.

Design changes before any participant

08-usability-testing/iteration-log.md states that no usability findings exist, so nothing in that log is a response to a test. The rows record rationale chosen while building: the trough definition, same-day ordering, no statistical range, a zero set-aside, an unsaved what-if, honest notification copy, and a layout fix so day rows and the 30-day table stay inside a 390px screen. The findings template’s EX-01 row is an example format, not a result.

Usability plan, not yet run

08-usability-testing/test-plan.md is ready and unused. The plan asks for five participants on a first concept test, including a keyboard-only session if possible. Five people are described as enough to see repeated blockers, and not as a sample that can stabilize the 70 percent target. Six tasks, plus an optional stale-balance task, are scripted: interpret the number, explain the low point, try a $150 dinner without saving it, add a monthly bill, remove it, and set a $250 alert while saying what happens if the page is closed.

Measures

Targets in 01-product-analysis/success-metrics.md, all unmeasured: at least 70 percent of prototype participants correctly interpret the forecast and the explanation; 60 percent of new test users add at least three items; 80 percent complete “Can I afford this?” without help; fewer than 10 percent believe a forecast is guaranteed. The prototype can measure fixture correctness and whether the scripted browser path completes. It cannot measure comprehension, activation, unaided task success, return visits, or any business metric. No business metric is defined. The prototype stores no analytics.

9. Outcomes, limitations, and next steps

Verified outcomes

- A reviewable case-study tree from product analysis through a usability plan, plus VALIDATION.md.
- Twelve annotated SVG wireframes and ten viewport screenshots of the running UI.
- A browser prototype that completes the primary path without a bank connection.
- Deterministic forecast behavior with 18 passing unit tests and a passing smoke script.
- Copy and structure that keep the estimate, the sample, and the scenario labeled.

Strengths of the work

The value proposition is inspectable: the number, the date, the items, and the formula are on real screens. Limits that the prototype cannot meet are written in the interface instead of simulated. The competitive frame is sourced and refuses claims the pages do not support. The research file is empty on purpose, which keeps the portfolio from presenting hypotheses as findings.

Limitations

- No evidence yet that the job, the segment, or the formula are understood by the people the brief describes.
- Manual entry may be too heavy. That risk is untested.
- localStorage is a launch blocker for real financial data.
- One pool, one symbol, no range, and no alert after the page closes.

- Accessibility is designed toward WCAG 2.2 AA and has not been audited.

Technical dependencies

Node runs the tests and the static server. TypeScript is a build-time dependency. The page itself is the compiled script plus app/styles.css. Persistence depends on the browser's localStorage. Fonts are requested from the page and fall back if blocked. There is no third-party data provider.

Highest-priority next steps

- Interview 5 to 8 people with 02-ux-research/interview-guide.md before treating the problem as confirmed.
- Run the usability sessions in 08-usability-testing/ with the prototype. Do not fill the findings template with expected results.
- Decide currency, retention, and whether a set-aside should ever be suggested.
- Replace localStorage before any live financial data is collected.
- Leave bank linking, household budgets, and savings goals out until the manual explanation survives a usability test.

No business impact, retention, or satisfaction result is claimed. None was measured.

10. Project artifact index

All paths are under 01-MoneyMind/ unless noted. 00-project-brief/ does not exist. The source requirements live one folder up, in 01-moneymind-predictive-finance.md.

Path	Contents
01-product-analysis/	Brief, problem, competitive analysis, positioning, assumptions, and success metrics.
02-ux-research/	Research plan, screener, interview guide, jobs, journey hypothesis, proto-personas, and an empty findings file.
03-information-architecture/	Sitemap, user flows, task analysis, and feature prioritization.
04-wireframes/	Screen inventory, specifications, annotations, and edge cases.
05-ui-design-system/	Principles, Clear ledger direction, tokens, components, and accessibility notes.
06-high-fidelity-ui/	Screen specifications, microcopy, and interaction specifications.
07-prototype/	Click path, interaction states, and a prototype review.
08-usability-testing/	Test plan, six scenarios, an empty findings template, and a pre-research iteration log.
09-case-study/	Narrative, slide outline, portfolio summary, this report, and the two-page overview.
app/	TypeScript prototype, compiled dist/, styles, forecast tests, and the browser smoke page.
assets/wireframes/	wf-01 through wf-12, annotated SVGs, plus the generator script.
assets/screenshots/	Ten viewport captures, 01-onboarding-mobile.png through 10-settings-mobile.png.
assets/diagrams/	safe-to-spend.svg, a diagram of the formula.
README.md	How to run the prototype, the folder guide, and the evidence status.
VALIDATION.md	Requirements coverage and the 22 October 2026 verification notes.
00-project-brief/	Missing. Use the parent requirements file named above.

11. Final status and evidence checklist

Deliverable	Evidence	Status	Remaining action
Product brief and UVP	01-product-analysis/product-brief.md, uvp-and-positioning.md	Complete	Keep the UVP labeled as a design position until interviews.
Competitor review	01-product-analysis/competitive-analysis.md	Complete	Re-open unverified prices before quoting them.
Research instruments	02-ux-research/research-plan.md, screener, interview guide	Partial	Recruit and run 5 to 8 interviews.
User findings	02-ux-research/research-findings.md	Missing	Leave empty until a real session.
Proto-personas and journey	personas.md, journey-map.md	Needs validation	Replace or retire after interviews.
Information architecture	03-information-architecture/	Complete	Revisit if the job is not a near-term shortfall.
Wireframes	assets/wireframes/wf-01 to wf-12	Complete	No Figma file exists or is claimed.
Design system	05-ui-design-system/	Complete	Light theme only. Contrast is calculated, not audited.
High-fidelity UI and prototype	app/, 06-high-fidelity-ui/, 07-prototype/	Complete	Run with npm start from app/.
Forecast correctness	app/test/forecast.test.ts	Complete	18 tests passed on 22 October 2026.

Deliverable	Evidence	Status	Remaining action
Primary browser path	app/test/smoke.html, VALIDATION.md	Complete	Smoke does not cover undo, export contents, or a screen reader.
Usability test	08-usability-testing/	Needs validation	Plan is ready. No participants.
Success metrics	01-product-analysis/success-metrics.md	Needs validation	Targets only. Do not report them as results.
Privacy and storage	#/privacy, localStorage key moneymind.v1	Partial	Disclosed. Not fit for live financial data.
WCAG 2.2 AA	05-ui-design-system/accessibility.md	Partial	Designed toward the target. Not certified.
Background reminders	#/reminders	Partial	In-app only. FR-06 is not fully met.
Bank linking	Feature prioritization, non-goals	Missing	Out of the MVP on purpose. Do not fake a connection.
00-project-brief/	Parent file 01-moneymind-predictive-finance.md	Missing	Requirements live outside this folder.

Evidence labels used in this report: verified means observed in the prototype, the test log, or a project file; documented means taken from a source named in the competitive analysis; assumed or hypothesis means a design position waiting for research; needs validation means a plan exists and no participant result exists.